DI31001 Group Project
IRC Bot and IRC Server
Code Functionality and Team Responsibilities

Group 2

October, 2026

1 Project Overview

This project implements a simplified IRC (Internet Relay Chat) system using IPv6 TCP. It contains two main programs:

- `server.py`: accepts multiple client connections, handles registration, manages channels, and routes channel and private messages.
- `bot.py`: connects to the server, joins a channel automatically, responds to commands, and replies to private messages.

Both programs use only the Python standard library. IRC messages are text-based and terminated by CRLF (`\r\n`).

2 System Architecture

The Ubuntu virtual machine runs the server and the Windows virtual machine runs the bot. HexChat can be used as a human IRC client. The programs communicate through IPv6 TCP.

```
HexChat client  -------- IPv6 TCP -------->  server.py
bot.py         -------- IPv6 TCP -------->  server.py
```

The default configuration is shown below.

Item	Default value
Server bind address	`::`
Server port	`6667`
Bot server address	`fc00:1337::17`
Bot nickname	`SuperBot`
Bot channel	`#test`

Table 1: Default network configuration

The communication sequence is:

1. The server opens an IPv6 TCP listening socket.
2. A client sends `NICK` and `USER` to register.
3. The server returns welcome messages, including numeric reply `001`.
4. The bot sends `JOIN` after receiving `001`.
5. Clients use `PRIVMSG` for channel and private messages.
6. `PING`/`PONG` messages maintain the connection.

3 Server Functionality (`server.py`)

3.1 Networking and Multiple Clients

The server uses an IPv6 TCP socket and Python's `selectors` event loop. Each connection is represented by a `ClientSession`. The server stores each client's nickname, username, real name, joined channels, and send/receive buffers. Non-blocking sockets allow multiple clients to be handled in one process.

The server also buffers incomplete IRC lines and limits line, receive-buffer, and send-buffer sizes to handle abnormal input safely.

3.2 Registration and Nickname Management

The server processes `NICK` and `USER`. After both are received, it marks the client as registered and sends the welcome and MOTD messages. It validates nicknames and prevents duplicate names:

- `431`: no nickname was supplied;
- `432`: the nickname is invalid;
- `433`: the nickname is already in use;
- `462`: a registered client attempted to register again.

3.3 Channel Management

`JOIN` creates a channel if needed, adds the member, and broadcasts the join event. `PART` removes the member and broadcasts the part event. `NAMES` returns the channel member list. Empty channels are removed.

Channel names must begin with `#` and cannot contain spaces, commas, or newline characters.

3.4 Message Routing

For `PRIVMSG`, a target beginning with `#` is treated as a channel and the message is sent to the other members. A nickname target is treated as a private message and is sent to the matching client. Unknown targets produce numeric error replies.

The sender does not receive a server echo for a channel message because IRC clients normally display their own messages locally.

The server also supports `NOTICE`, `WHO`, `MODE`, and basic `CAP` negotiation used by HexChat.

3.5 Errors and Keep-Alive

Unknown commands produce numeric reply `421`. Malformed input is isolated to the affected client. The server periodically checks activity, sends `PING` to idle clients, and disconnects clients that do not return `PONG`. Nickname and channel state are cleaned up on disconnection.

4 Bot Functionality (`bot.py`)

4.1 Connection and Registration

The bot creates an IPv6 TCP socket using `AF_INET6` and `SOCK_STREAM`. It sends:

```
NICK SuperBot
USER SuperBot 0 * :SuperBot coursework bot
```

After receiving `001`, it joins its configured channel. It automatically reconnects after a connection failure. If the nickname is taken, it appends an underscore and retries.

4.2 Member List

The bot maintains a set of channel members by processing `353`, `JOIN`, `PART`, `NICK`, and `QUIT`. This set is used by the random `!slap` command.

4.3 Channel Commands

Table 2: Bot channel commands

Command	Function
`!hello`	Greets the sender.
`!slap`	Selects a random member, excluding the bot and sender. If no other member exists, the sender is selected.
`!slap <nickname>`	Slaps the named user if present; otherwise slaps the sender.
`!me <text>`	Sends a CTCP ACTION message.
`!who`	Uses `WHO` and reports the number of returned member records.

4.4 Private Messages

When the target matches the bot's nickname, the bot selects a random non-empty line from `facts.txt` and replies with `PRIVMSG`. If the file is missing or empty, a built-in fallback reply is used.

4.5 Command-Line Options

```
python bot.py --host <IPv6 address> --port <port> \
    --name <nickname> --channel <channel>
```

These options allow the same program to run on the virtual machines or a local loopback address.

5 Mapping to the Marking Criteria

This section explains how the implemented bot functionality satisfies the marking criteria.

5.1 Item E: Connecting, Handling Errors, and Staying Connected

The bot must connect to the IRC server, maintain the connection, and handle common connection errors.

The implementation satisfies this requirement in the following ways:

- The bot creates an IPv6 TCP socket using `AF_INET6` and `SOCK_STREAM`.
- The bot connects to the configured server address and port.
- It sends `NICK` and `USER` to complete registration.
- It responds to server `PING` messages with `PONG`.
- If the connection is lost, the bot closes the old socket and attempts to reconnect automatically.
- If the nickname is already in use, the bot appends an underscore and tries again.
- Connection failures are caught and handled without terminating the whole program.

The main implementation is provided by the following methods:

```
connect()
run()
_handle()
```

A simple test is to stop the server after the bot has connected and then restart it. The bot should enter its reconnection loop and connect again when the server becomes available.

5.2 Item F: Joining a Channel and Maintaining the User List

The bot must join a channel and keep track of the users currently in that channel.

After receiving the registration welcome message `001`, the bot automatically sends a `JOIN` command for the configured channel.

The bot updates its member list when it receives the following IRC messages:

- `353`: initial channel names list;
- `JOIN`: a user joins the channel;
- `PART`: a user leaves the channel;
- `NICK`: a user changes nickname;
- `QUIT`: a user disconnects from the server.

The member list is stored in the bot's `users` set. It is mainly used by the `!slap` command to select a valid target.

The main implementation is provided by:

```
_update_users_from_names()
_handle()
```

5.3 Item G: Replying to Private Messages

The bot must reply to every private message with a random sentence, fact, or nonsense message.

When the target of a `PRIVMSG` matches the bot's nickname, the bot treats the message as a private message. It selects one non-empty line from `facts.txt` and sends it back to the sender.

For example:

```
Alice -> TestBot: Hello
TestBot -> Alice: Honey never spoils.
```

The main implementation is provided by:

```
_reply_private()
load_facts()
```

If `facts.txt` is missing or empty, the bot uses a built-in fallback response instead of crashing.

This feature can be tested in HexChat by opening a private chat with the bot and sending any message.

5.4 Item H: Channel Commands

The bot must respond to channel messages beginning with an exclamation mark. The required commands are `!hello` and `!slap`.

5.4.1 `!hello`

The command:

```
!hello
```

causes the bot to send a greeting to the user who issued the command. For example:

```
Hello Alice! Nice to meet you.
```

5.4.2 `!slap`

The command:

```
!slap
```

selects a random channel member. The bot excludes itself and the user who issued the command whenever other candidates are available.

The command also accepts an optional nickname:

```
!slap Alice
```

If Alice is present in the channel, Alice is selected. If Alice is not present, the sender of the command is selected instead.

The main implementation is provided by:

```
_handle_command()
_do_slap()
```

The bot sends the result as a CTCP ACTION message, which is displayed by IRC clients as an action or emote.

5.5 Item I: Additional IRC Features

The project implements two additional features beyond the required commands.

5.5.1 `!me`: CTCP ACTION

The command:

```
!me waves
```

is converted into a CTCP ACTION message. An IRC client may display the result as:

```
* TestBot waves
```

This feature uses the CTCP ACTION format inside a `PRIVMSG` message.

5.5.2 `!who`: WHO Command

The command:

```
!who
```

causes the bot to send the IRC `WHO` command for the current channel.

The server returns one or more `352` WHO reply records followed by a `315` end-of-list message. The bot counts the returned records and reports the result in the channel.

For example:

```
WHO returned 2 member record(s) for #test.
```

The main implementation is provided by:

```
_report_who()
```

These features demonstrate the use of IRC functionality beyond basic channel messages and private messages.

5.6 Item J: Supporting Multiple Clients

The server must allow multiple IRC clients to connect at the same time and manage them correctly.

The server creates one `ClientSession` object for each connected client. It stores each session in an internal session table and uses the `selectors` event loop to monitor all sockets.

The implementation supports:

- multiple simultaneous TCP connections;
- independent registration for each client;
- separate nicknames, usernames, buffers, and channel memberships;
- communication between multiple HexChat clients and the bot;
- correct removal of a client after disconnection.

The main implementation is provided by:

```
IRCServer.start()
_accept_client()
ClientSession
```

This feature can be tested by connecting two or more HexChat clients to the same server using different nicknames.

5.7 Item K: Error Handling

The server must handle unknown commands, invalid nicknames, malformed messages, and disconnected clients without crashing.

The server provides the following error handling:

- unknown commands return numeric reply `421`;
- invalid nicknames return `432`;
- duplicate nicknames return `433`;
- missing command parameters return the appropriate numeric error;
- messages sent to nonexistent users or channels return an error;
- malformed input is isolated to the affected client;
- clients that stop responding are detected and disconnected;
- disconnected clients are removed from all channels and internal tables.

The main implementation is provided by:

```
parse_irc_line()
_dispatch()
_disconnect()
_check_liveness()
```

This ensures that an invalid request from one client does not terminate the whole server.

5.8 Item L: Private Messaging

The server must support private messages between individual clients.

When a client sends a `PRIVMSG` with a nickname as the target, the server searches its nickname index and forwards the message only to that client.

For example:

```
Alice -> Bob: Hello Bob
```

Only Bob receives the message. Other clients and channels do not receive it.

If the target nickname does not exist, the server returns numeric reply `401`.

The main implementation is provided by:

```
_cmd_privmsg()
_route_message()
```

This feature can be tested by connecting two HexChat clients and sending a private message from one client to the other.

5.9 Item M: Joining and Leaving Channels

The server must support clients joining and leaving channels and must notify other channel members.

The `JOIN` command performs the following operations:

- validates the channel name;
- creates the channel if it does not already exist;
- adds the client to the channel;
- broadcasts a `JOIN` message to the channel;
- sends the topic and member list replies.

The `PART` command:

- removes the client from the channel;
- broadcasts a `PART` message;
- returns an error if the client is not in the channel;
- removes the channel when it has no remaining members.

The main implementation is provided by:

```
_cmd_join()
_cmd_part()
_send_names()
```

This feature can be tested in HexChat with:

```
/join #test
/part #test
```

Other users in the channel should see the join and part events.

5.10 Item N: Channel Messaging

The server must support messages sent to all members of a channel.

When a client sends a `PRIVMSG` whose target begins with `#`, the server checks whether the client is a member of that channel. If the client is a member, the message is broadcast to the other members of the channel.

For example:

```
Alice -> #test: Hello everyone
```

All other members of `#test` receive the message. Alice does not receive a duplicate server echo because her IRC client already displays her own message locally.

If the channel does not exist, the server returns `403`. If the sender is not a member of the channel, the server returns `404`.

The main implementation is provided by:

```
_cmd_privmsg()
_route_message()
_broadcast_channel()
```

This feature can be tested by connecting at least two clients, joining the same channel, and sending a normal message.

6 Running and Testing

6.1 Two Virtual Machines

On Ubuntu, start the server:

```
python3 server.py
```

On Windows, start the bot:

```
python bot.py
```

Connect with HexChat and enter:

```
/join #test
```

The following commands can then be entered in the channel:

```
!hello
!slap
!slap Alice
!me waves
!who
```

Sending a private message to the bot tests the random fact reply.

6.2 Local Testing

For local testing on one computer, use the IPv6 loopback address:

```
python server.py --host ::1 --port 16667
python bot.py --host ::1 --port 16667 --name TestBot --channel "#local"
```

In HexChat, use server address `::1`, port `16667`, and disable SSL. Join `#local`, then run the commands in Table 2.

7 Team Responsibilities

The following table is intentionally blank. Fill it in with the group's actual names, student numbers, and contributions.

Table 3: Team responsibilities

Name/Student ID	Module	Specific contribution
Zhuoyuan He / 2666094		

Name/Student ID	Module	Specific contribution
Zheng Gong / 2666141		
Ruihang Gao / 2666078		
Qiwei Zhao / 2666205		
Daran Deng / 2666127		

Possible areas include server implementation, bot implementation, IPv6 and virtual-machine configuration, HexChat testing, documentation, and demonstration preparation. Each entry should reflect actual work.

8 Project File List

File	Purpose
`server.py`	IPv6 IRC server source code.
`bot.py`	IRC bot client source code.
`facts.txt`	Private-message replies, one fact per line.
`README.md`	Basic project and run instructions.
`marking-sheet.pdf`	The completed course marking sheet.

Table 4: Recommended submission files

9 Current Status and Future Work

The current version implements IPv6 connections, IRC registration, channel joining and leaving, channel messages, private messages, bot commands, random fact replies, the WHO extension, PING/PONG keep-alive, and automatic reconnection.

Before submission, the team should add the actual member information, record test results or screenshots, and check each item against the marking sheet.